

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
DO ESTADO DE MINAS GERAIS
CAMPUS DIVINÓPOLIS

ENGENHARIA DE COMPUTAÇÃO

SISTEMAS OPERACIONAIS

Manual da Máquina (Assembly)

04 de Outubro, 2026



Sumário

1	Apresentação e Introdução	2
2	Visão Geral da Arquitetura	2
2.1	Paradigma Load/Store	2
3	Formato da Instrução (16 bits)	2
3.1	Campos de Decodificação	2
4	Tabela de Opcodes	3
5	Codificação Detalhada por Instrução	3
5.1	LOAD — Carregar da Memória	3
5.2	ADD — Soma	3
5.3	STORE — Salvar na Memória	4
5.4	SUB — Subtração	4
5.5	AND — E lógico (bit a bit)	4
5.6	OR — OU lógico (bit a bit)	4
5.7	BEQ — Branch if Equal (Desvio Condicional)	4
5.8	JUMP — Salto Incondicional	4
5.9	HALT — Encerrar Execução	4
6	Formato do Arquivo de Job (.txt)	5
6.1	Estrutura	5
6.2	Regras do Parser	5
6.3	Endereçamento	5
7	Exemplos Completos	6
7.1	Job 1 — Soma Simples ($C = A + B$)	6
7.2	Job 2 — Operações Aritméticas e Lógicas	6
7.3	Job 3 — Laço de Repetição (Contagem Regressiva)	6
8	Mapa de Memória	7
9	Erros Comuns e Dicas	8
9.1	Armadilha do BEQ	8

1 Apresentação e Introdução

O presente documento apresenta a documentação oficial do conjunto de instruções (ISA), do formato de codificação e das regras do Assembly utilizado pelo Simulador de Sistemas Operacionais em processamento batch.

2 Visão Geral da Arquitetura

Tabela 1: Características da arquitetura

Característica	Valor
Tamanho da palavra	16 bits
Paradigma	RISC Load/Store
Registradores gerais	8 (R0–R7), cada um de 16 bits
Registradores especiais	PC (Program Counter), IR (Instruction Register)
Tamanho da RAM	65.536 palavras ($64K \times 16$ bits)
Área reservada ao SO	Endereços 0x0000 – 0x01FF (0 a 511)
Base do programa	Endereço 0x0200 (512 em decimal)

2.1 Paradigma Load/Store

Todas as operações aritméticas e lógicas ocorrem **exclusivamente entre registradores**. Dados precisam ser trazidos da memória via **LOAD** antes de qualquer cálculo, e os resultados são devolvidos via **STORE**.

```
LOAD R0, A      ; Traz A da RAM -> R0
LOAD R1, B      ; Traz B da RAM -> R1
ADD  R2, R0, R1 ; R2 = R0 + R1 (ULA)
STORE R2, C     ; Salva R2 -> RAM[C]
HALT            ; Fim
```

3 Formato da Instrução (16 bits)

Cada instrução é codificada em uma única palavra de 16 bits, dividida em 4 nibbles (4 bits cada):

15–12	11–8	7–4	3–0
Opcode	Rd	Rf1	Rf2

3.1 Campos de Decodificação

Tabela 2: Campos de decodificação

Campo	Bits	Máscara	Descrição
opcode	15–12	(IR » 12) & 0xF	Código da operação
regDest	11–8	(IR » 8) & 0xF	Registrador de destino (ou 1º operando em BEQ)
regF1	7–4	(IR » 4) & 0xF	1º registrador fonte (ULA) ou parte do endereço
regF2	3–0	IR & 0xF	2º registrador fonte (ULA) ou parte do endereço
endereço	7–0	IR & 0xFF	Endereço relativo de 8 bits (LOAD/STORE/BEQ)

Atenção: Os campos **regF1/regF2** e **endereço** compartilham os mesmos bits (7–0). A interpretação depende do opcode:

- **Instruções ULA** (ADD, SUB, AND, OR): usam **regF1** e **regF2** como registradores.
- **LOAD/STORE**: usam **endereço** (8 bits).
- **BEQ**: usa **regDest** e **regF1** como registradores de comparação, e **endereço** (8 bits) como alvo do salto.
- **JUMP**: usa 12 bits (bits 11–0) como endereço, via **IR & 0x0FFF**.

4 Tabela de Opcodes

Tabela 3: Tabela de opcodes

Mnemônico	Opcode	Hex	Categoria	Formato	Semântica
LOAD	0001	0x1	Transferência	1 Rd Addr[7:0]	$R[Rd] = RAM[Addr + 512]$
ADD	0010	0x2	ULA (Aritmética)	2 Rd Rf1 Rf2	$R[Rd] = R[Rf1] + R[Rf2]$
STORE	0011	0x3	Transferência	3 Rd Addr[7:0]	$RAM[Addr + 512] = R[Rd]$
SUB	0100	0x4	ULA (Aritmética)	4 Rd Rf1 Rf2	$R[Rd] = R[Rf1] - R[Rf2]$
AND	0101	0x5	ULA (Lógica)	5 Rd Rf1 Rf2	$R[Rd] = R[Rf1] \& R[Rf2]$
OR	0110	0x6	ULA (Lógica)	6 Rd Rf1 Rf2	$R[Rd] = R[Rf1] R[Rf2]$
BEQ	0111	0x7	Controle de Fluxo	7 Rd Rf1 Low	$R[Rd] == R[Rf1], \quad PC = Addr + 512$
JUMP	1000	0x8	Controle de Fluxo	8 Addr[11:0]	$PC = (IR \& 0x0FFF) + 512$
HALT	1111	0xF	Sistema	F 000	Encerra a execução do Job

Nota sobre BEQ: O endereço de salto é **IR & 0xFF** (8 bits), que **sobrepõe** o nibble de **regF1**. Isso significa que o nibble alto do endereço-alvo é o **mesmo** que o índice do registrador fonte 1. Ao codificar um BEQ, deve-se compatibilizar o registrador de comparação com o endereço-alvo desejado.

5 Codificação Detalhada por Instrução

5.1 LOAD — Carregar da Memória

Formato: 1 Rd AA
 Exemplo: 100A -> LOAD R0, [0x0A]
 $R0 = RAM[0x0A + 512] = RAM[522]$

5.2 ADD — Soma

Formato: 2 Rd Rf1 Rf2
 Exemplo: 2201 -> ADD R2, R0, R1
 $R2 = R0 + R1$

5.3 STORE — Salvar na Memória

```
Formato: 3 Rd AA
Exemplo: 320C -> STORE R2, [0x0C]
          RAM[0x0C + 512] = R2
```

5.4 SUB — Subtração

```
Formato: 4 Rd Rf1 Rf2
Exemplo: 4201 -> SUB R2, R0, R1
          R2 = R0 - R1
```

5.5 AND — E lógico (bit a bit)

```
Formato: 5 Rd Rf1 Rf2
Exemplo: 5301 -> AND R3, R0, R1
          R3 = R0 & R1
```

5.6 OR — OU lógico (bit a bit)

```
Formato: 6 Rd Rf1 Rf2
Exemplo: 6301 -> OR R3, R0, R1
          R3 = R0 | R1
```

5.7 BEQ — Branch if Equal (Desvio Condicional)

```
Formato: 7 Rd Rf1 Low_nibble
          endereco = (Rf1 << 4) | Low_nibble = IR & 0xFF
Exemplo: 7308 -> BEQ R3, R0, endereco=0x08
          Se R3 == R0 -> PC = 0x08 + 512
```

Cuidado: O nibble Rf1 é simultaneamente o índice do registrador e o nibble alto do endereço-alvo. Para saltar para o endereço 0x08 comparando com R0, o registrador de comparação **DEVE** ser R0 (nibble = 0).

5.8 JUMP — Salto Incondicional

```
Formato: 8 Addr[11:0]
          PC = (IR & 0x0FFF) + 512
Exemplo: 8003 -> JUMP 0x003
          PC = 0x003 + 512 = 515
```

5.9 HALT — Encerrar Execução

```
Formato: F 000
Exemplo: F000 -> HALT
          Define haltStatus = true; CPU para de executar.
```

6 Formato do Arquivo de Job (.txt)

Os programas em assembly são escritos em arquivos `.txt` com o seguinte formato:

6.1 Estrutura

```
// Comentários começam com //  
// Tudo após // em uma linha é ignorado  
  
<instrução_hex> // Comentário opcional  
<instrução_hex> // Comentário opcional  
...  
  
.data  
<endereço_relativo> <valor_hex> // Variável  
<endereço_relativo> <valor_hex> // Variável  
...
```

6.2 Regras do Parser

1. **Comentários:** Linhas iniciadas com `//` ou conteúdo após `//` são ignorados.
2. **Espaços:** Espaços em branco antes e depois do conteúdo são removidos (trim).
3. **Linhas vazias:** São ignoradas.
4. **Seção `.data`:** Marca o início da declaração de variáveis.
5. **Seção `.text`:** (Opcional) Marca o início das instruções (padrão).
6. **Instruções:** Valores hexadecimais de 4 dígitos (16 bits) na seção de texto.
7. **Variáveis:** Formato `<endereço_relativo> <valor>`, ambos em hexadecimal.

6.3 Endereçamento

Todos os endereços nos arquivos `.txt` são **relativos à base 512** (0x0200). O Loader/RAMLoader soma automaticamente `OS_RESERVED_MEM = 512` ao endereço relativo para obter o endereço absoluto na RAM.

Tabela 4: Conversão de endereços relativos para absolutos

Endereço no <code>.txt</code>	Endereço absoluto na RAM	Fórmula
0x00	0x0200 (512)	$0x00 + 512 = 512$
0x0A	0x020A (522)	$0x0A + 512 = 522$
0x0C	0x020C (524)	$0x0C + 512 = 524$
0xFF	0x02FF (767)	$0xFF + 512 = 767$

7 Exemplos Completos

7.1 Job 1 — Soma Simples ($C = A + B$)

```
// Soma de A e B (Seção 4 do PDF da disciplina)
100A // LOAD R0, [0x0A] -> R0 = A = 5
110B // LOAD R1, [0x0B] -> R1 = B = 7
2201 // ADD R2, R0, R1 -> R2 = 5 + 7 = 12 (0x000C)
320C // STORE R2, [0x0C] -> RAM[C] = 12
F000 // HALT

.data
0A 0005 // A = 5
0B 0007 // B = 7
0C 0000 // C = 0 (resultado)
```

Resultado esperado: R0=0x0005, R1=0x0007, R2=0x000C.

7.2 Job 2 — Operações Aritméticas e Lógicas

```
// Subtração e AND com múltiplos registradores
100D // LOAD R0, [0x0D] -> R0 = X = 20 (0x0014)
110E // LOAD R1, [0x0E] -> R1 = Y = 8 (0x0008)
4201 // SUB R2, R0, R1 -> R2 = 20 - 8 = 12 (0x000C)
5301 // AND R3, R0, R1 -> R3 = 0x0014 & 0x0008 = 0x0000
320F // STORE R2, [0x0F] -> Salva resultado da SUB
F000 // HALT

.data
0D 0014 // X = 20
0E 0008 // Y = 8
0F 0000 // Resultado
```

Resultado esperado: R0=0x0014, R1=0x0008, R2=0x000C, R3=0x0000.

7.3 Job 3 — Laço de Repetição (Contagem Regressiva)

```
// Loop: Contador 3->2->1->0 usando BEQ e JUMP
100A // LOAD R0, [0x0A] -> R0 = 0 (constante de comparação)
110B // LOAD R1, [0x0B] -> R1 = 1 (decremento)
130C // LOAD R3, [0x0C] -> R3 = 3 (contador)
7308 // BEQ R3, R0, 0x08 -> Se R3==0, salta para HALT
4331 // SUB R3, R3, R1 -> R3 = R3 - 1
330C // STORE R3, [0x0C] -> Salva Contador atualizado
8003 // JUMP 0x003 -> Volta ao BEQ
330C // STORE R3, [0x0C] -> Salva resultado final (0)
F000 // HALT

.data
0A 0000 // Zero (constante)
0B 0001 // Decremento = 1
0C 0003 // Contador = 3
```

Resultado esperado: R0=0x0000, R1=0x0001, R3=0x0000 (3 iterações do laço).

Trace de execução (fluxo de controle):

```

Iteração 1: R3=3 != 0 -> SUB -> R3=2 -> STORE -> JUMP -> volta ao BEQ
Iteração 2: R3=2 != 0 -> SUB -> R3=1 -> STORE -> JUMP -> volta ao BEQ
Iteração 3: R3=1 != 0 -> SUB -> R3=0 -> STORE -> JUMP -> volta ao BEQ
Saída:      R3=0 == 0 -> BEQ salta para HALT -> Fim

```

8 Mapa de Memória

A memória principal (RAM) do simulador possui capacidade de 65.536 palavras de 16 bits (endereçáveis de 0x0000 a 0xFFFF). A organização do espaço de endereçamento é dividida em duas áreas principais, como ilustrado na Figura 1:

- **Área Reservada ao Sistema Operacional (0x0000 a 0x01FF):** Compreende as primeiras 512 palavras da memória (endereços 0 a 511 em decimal). Esta região é de uso exclusivo do Kernel/SO. Qualquer tentativa de acesso direto por um programa de usuário (*Job*) a este espaço dispara uma exceção de violação de memória (SEGFAULT).
- **Área de Usuário / Programa (0x0200 a 0xFFFF):** Inicia exatamente no endereço decimal 512 (0x0200). É onde os programas são carregados e executados. As instruções da seção `.text` ocupam as posições iniciais a partir da base, seguidas pelas variáveis declaradas na seção `.data`.

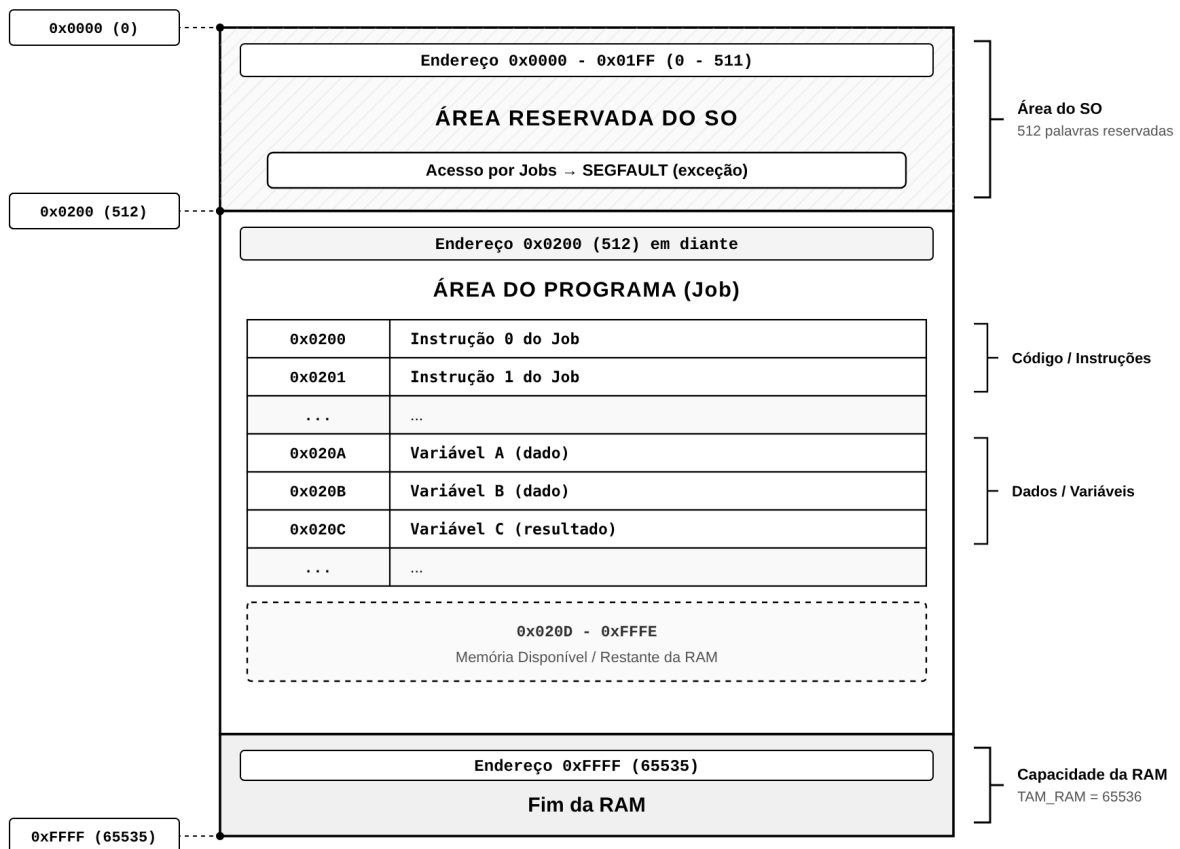


Figura 1: Organização e divisão do mapa de memória RAM do simulador.

9 Erros Comuns e Dicas

Tabela 5: Erros comuns e dicas

Erro	Causa	Solução
SEGFAULT: Tentativa de leitura em area restrita	Job tentou acessar endereço < 512	Verificar endereços nos LOAD/STORE
Opcode Invalido	Nibble do opcode não corresponde a nenhuma instrução	Verificar codificação hexadecimal
Erro ao converter instrucao	Linha na seção .text não é hex válido	Verificar formato (4 dígitos hex)
Job nunca termina (loop infinito)	BEQ/JUMP mal codificado ou sem HALT	O BatchManager aborta após 1M instruções

9.1 Armadilha do BEQ

O campo de endereço do BEQ (8 bits) **inclui** o nibble do registrador fonte 1. Para saltar para o endereço 0x08 comparando com R0:

- ✓ 7308 → regDest=R3, regF1=R0, endereço=0x08
- × 7328 → regDest=R3, regF1=R2, endereço=0x28 (salto para endereço errado!)